



Module 7 – Sécurité

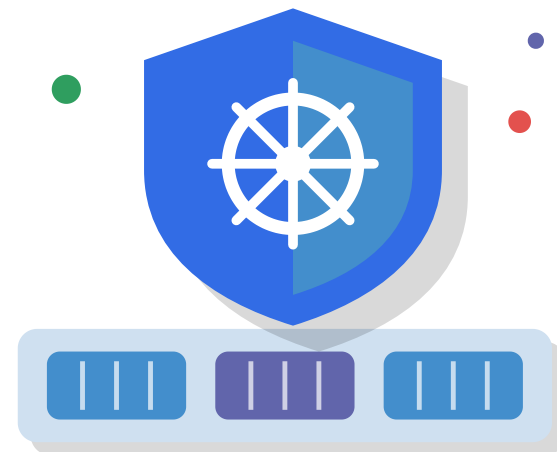
Protéger le cluster et l'application TrackR

Kubernetes – Orchestration des conteneurs

Objectifs du module

À la fin de ce module, vous saurez :

- identifier la **surface d'attaque** d'un cluster ;
- contrôler l'**accès à l'API** : identité, RBAC, ServiceAccounts ;
- imposer des Pods sûrs avec **Pod Security Admission** ;
- plafonner les ressources d'un namespace ;
- cloisonner le réseau avec des **NetworkPolicy** ;
- protéger **etcd** et chiffrer les Secrets au repos.



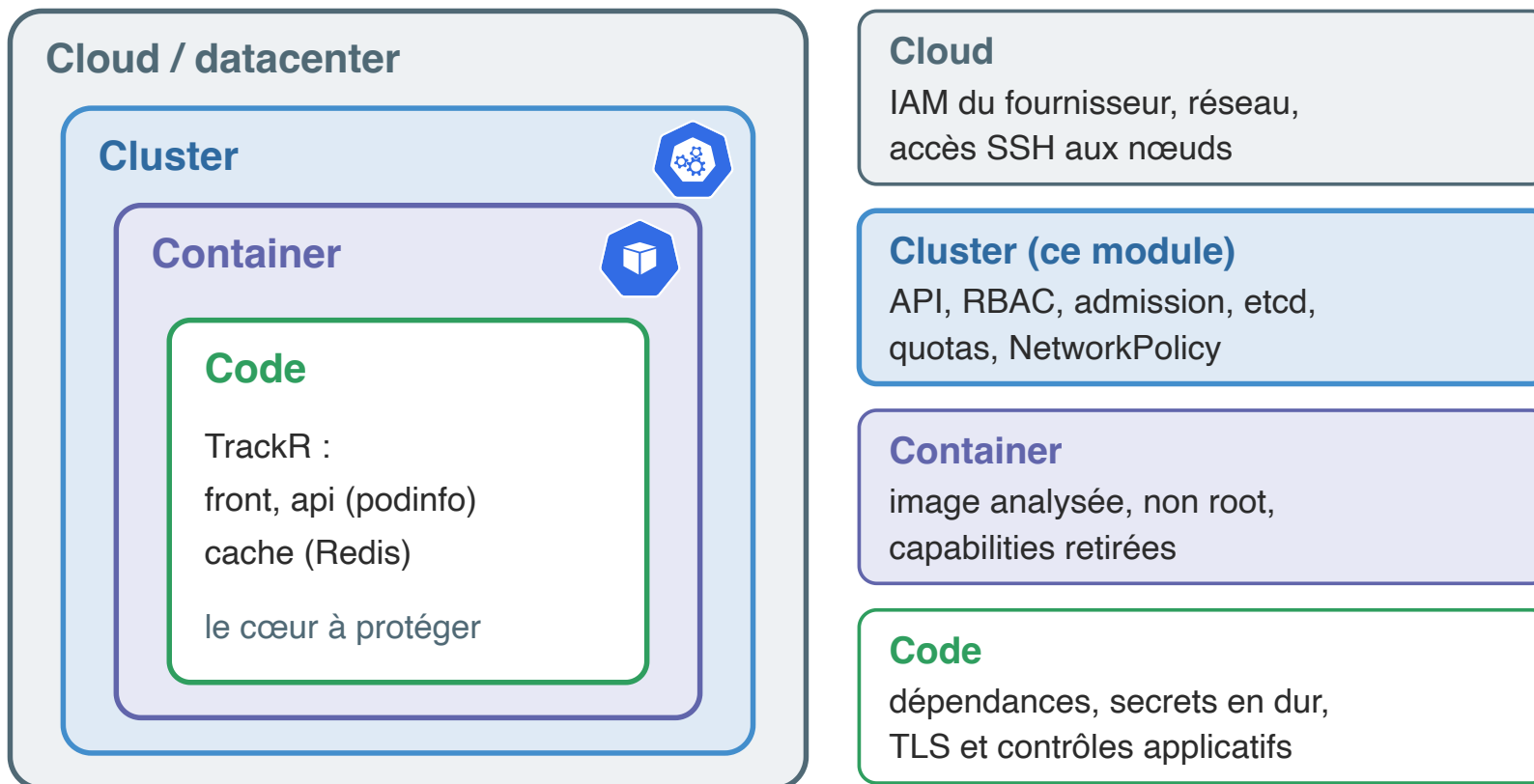
Plan du module

1. Points à sécuriser : les 4C et la surface d'attaque
2. Accès à l'API : authentication, RBAC, ServiceAccounts
3. Admission : Pod Security Standards et `securityContext`
4. Ressources, réseau et etcd : quotas, NetworkPolicy, chiffrement
5. Outils pour aller plus loin

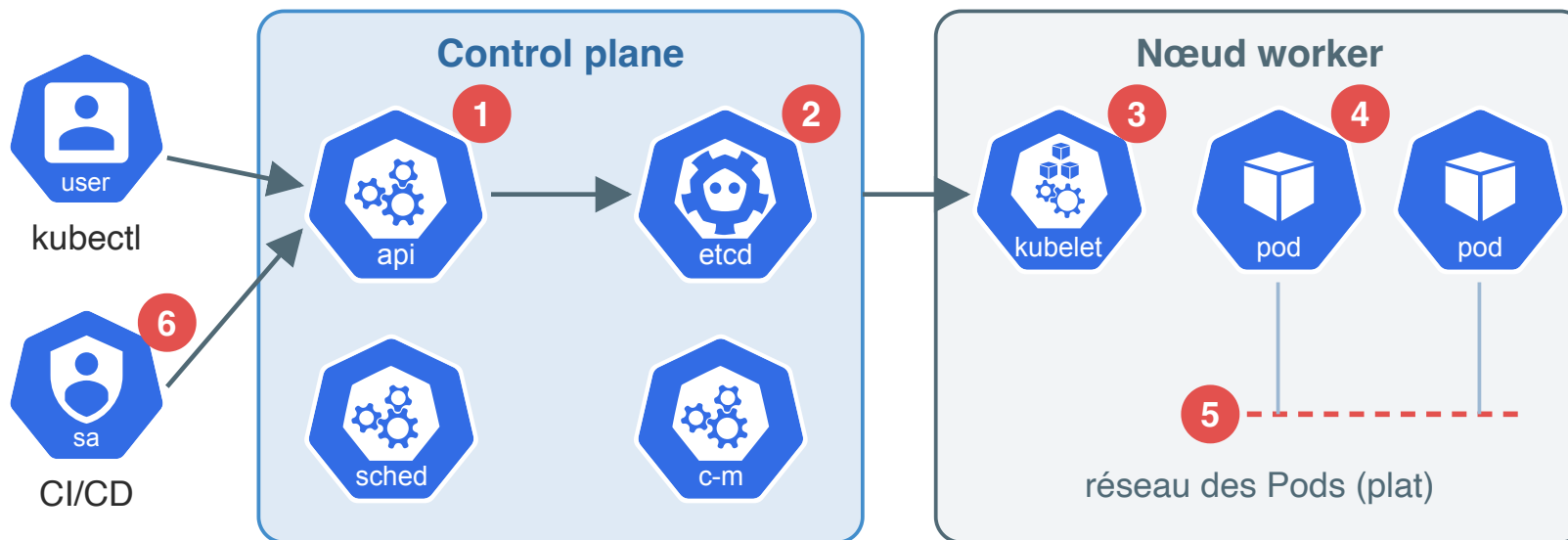
Puis la **Démo 7** (RBAC, NetworkPolicy, etcd) et le **TP 7** : sécuriser le namespace TrackR.

1. Points à sécuriser

Les 4C de la sécurité cloud native



La surface d'attaque d'un cluster



1 API exposée : identité, RBAC

2 etcd : tous les Secrets du cluster

3 API du kubelet (port 10250)

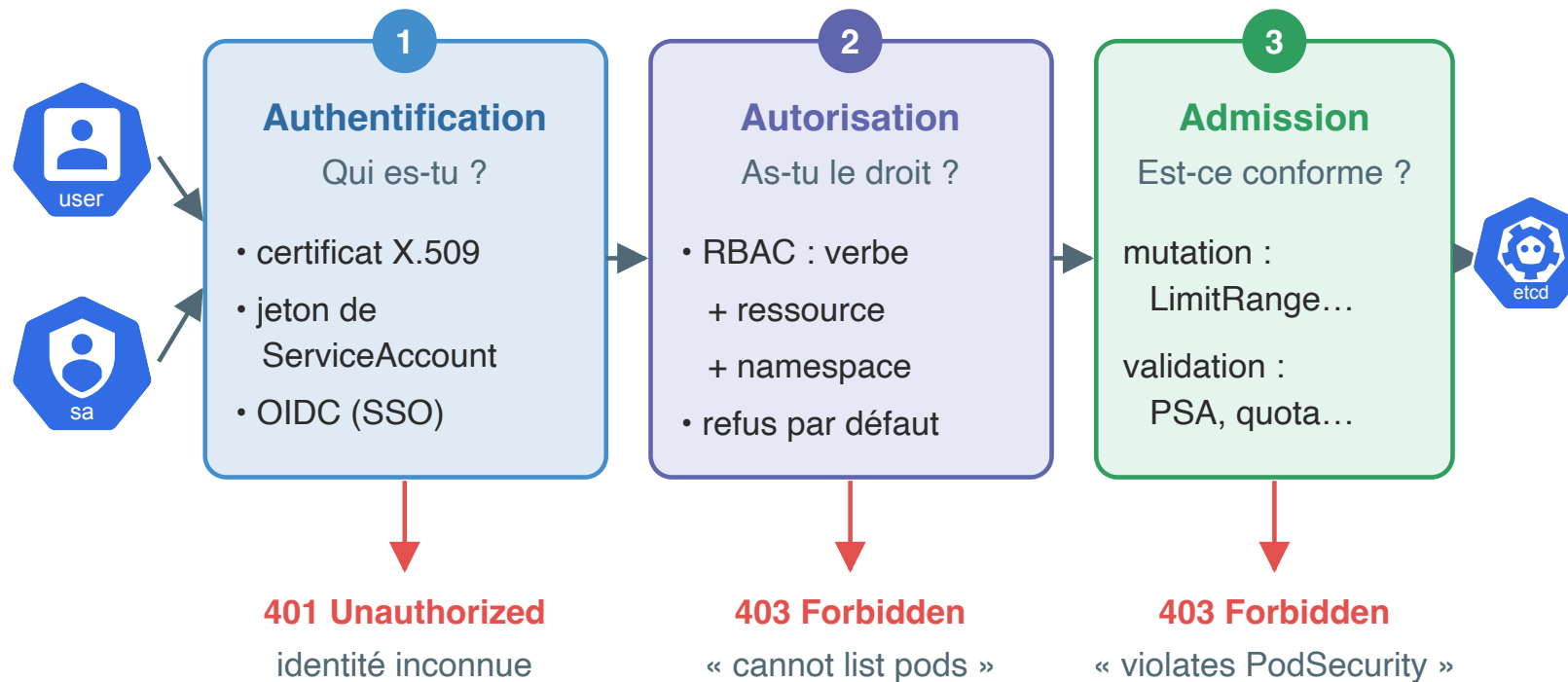
4 Pod root, privilégié, image vulnérable

5 tout Pod joint tout Pod

6 jetons trop puissants ou éternels

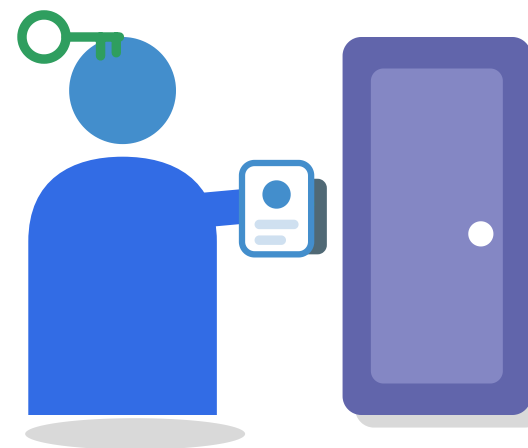
2. Accès à l'API Kubernetes

Authentication, autorisation, admission

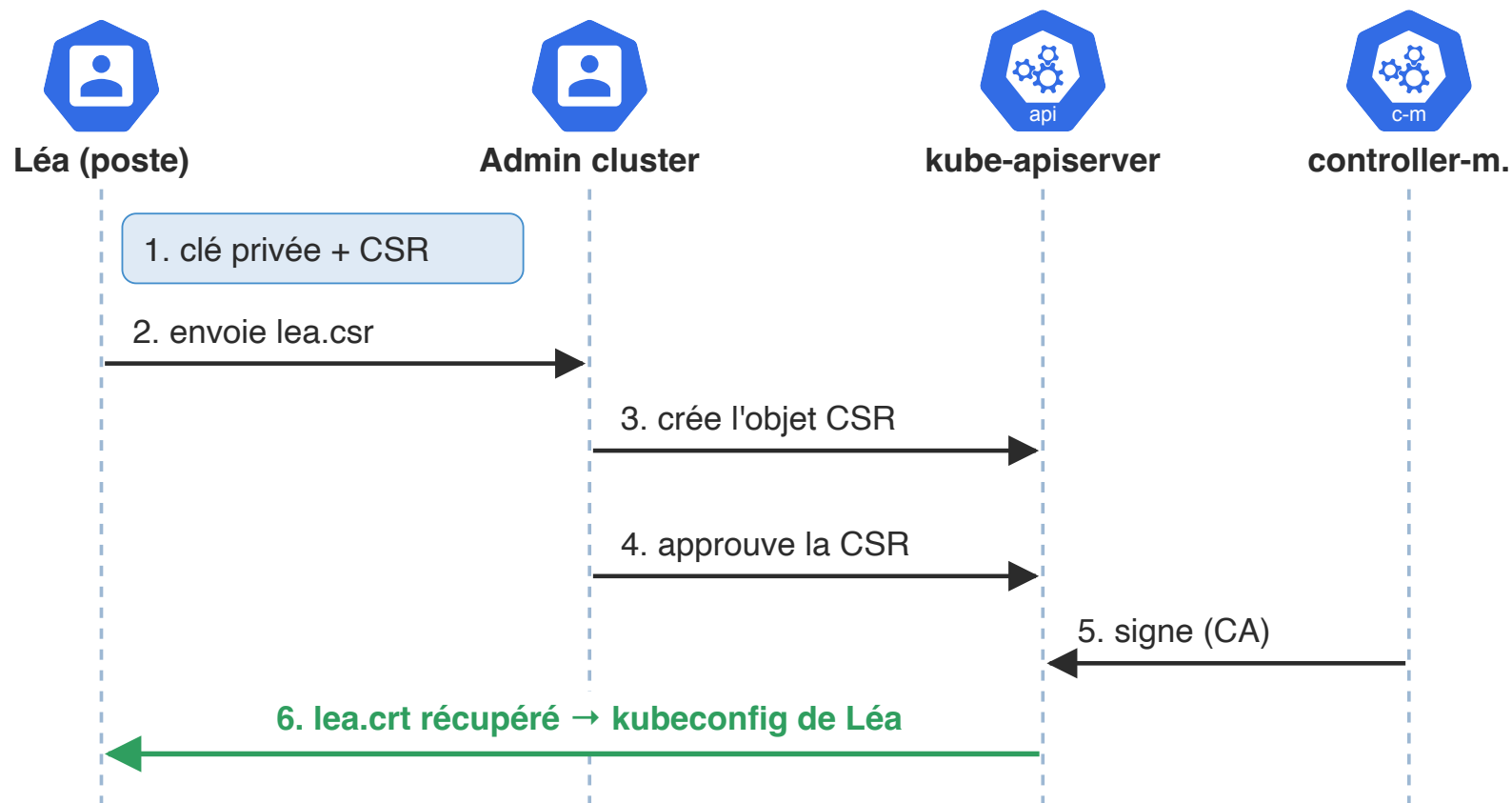


Authentification : qui es-tu ?

- Kubernetes ne stocke **aucun compte utilisateur** : il fait confiance à des émetteurs d'identité.
- **Certificat X.509** signé par la CA du cluster : `CN` = utilisateur, `O` = groupe.
- **Jeton de ServiceAccount** : identité des Pods et des pipelines.
- **OIDC** : SSO d'entreprise (Keycloak, Entra ID...), non rejoué sur le lab.
- `kubectl auth whoami` montre l'identité vue par l'API.



Créer un utilisateur : la CSR



La CSR de Léa, rejouée sur le lab

```
openssl genrsa -out lea.key 2048
openssl req -new -key lea.key -out lea.csr -subj "/CN=lea/O=logivia-dev"
kubectl apply -f 10-csr-lea.yaml          # request = base64 de lea.csr, expirationSeconds 86400
kubectl certificate approve lea
kubectl get csr lea -o jsonpath='{.status.certificate}' | base64 -d > lea.crt
```

```
$ kubectl get csr lea
NAME    AGE    SIGNERNAME              REQUESTOR    REQUESTEDDURATION   CONDITION
lea     1s     kubernetes.io/kube-apiserver-client  kubernetes-admin    24h                 Approved,Issued
$ openssl x509 -in lea.crt -noout -subject -issuer -dates
subject=O=logivia-dev, CN=lea
issuer=CN=kubernetes
notBefore=Oct  4 20:48:48 2026 GMT
notAfter=Oct  5 20:48:48 2026 GMT
```

La clé privée ne quitte jamais le poste de Léa ; le certificat expire au bout de 24 h.

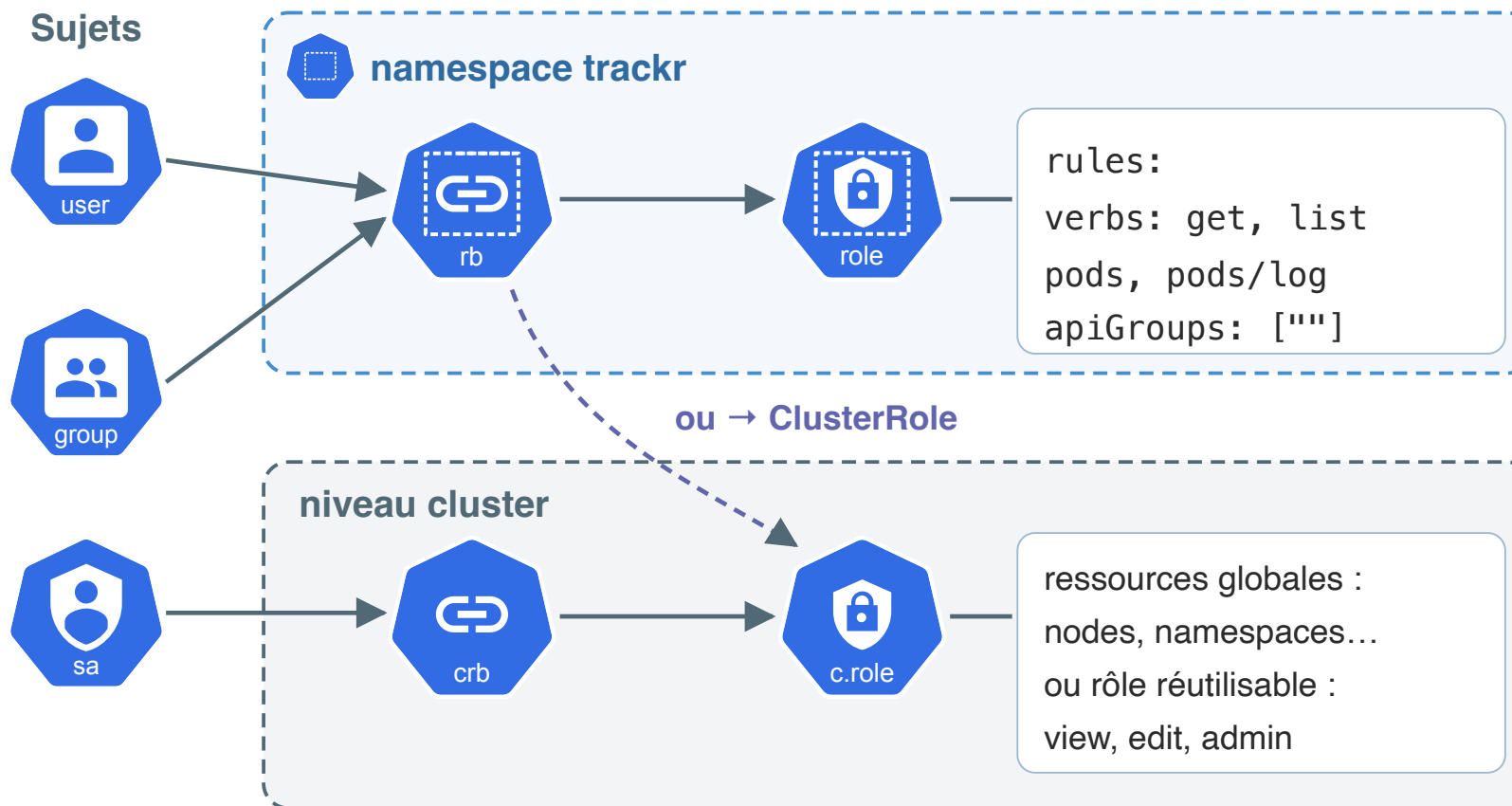
Un kubeconfig dédié à Léa

```
SERVER=$(kubectl config view --minify -o jsonpath='{.clusters[0].cluster.server}')
kubectl config set-cluster k8s-formation --server=$SERVER --certificate-authority=ca.crt \
  --embed-certs --kubeconfig=lea.kubeconfig
kubectl config set-credentials lea --client-certificate=lea.crt --client-key=lea.key \
  --embed-certs --kubeconfig=lea.kubeconfig
kubectl config set-context lea@k8s-formation --cluster=k8s-formation --user=lea \
  --namespace=trackr --kubeconfig=lea.kubeconfig
kubectl config use-context lea@k8s-formation --kubeconfig=lea.kubeconfig
```

```
$ kubectl --kubeconfig=lea.kubeconfig auth whoami
ATTRIBUTE                                VALUE
Username                                lea
Groups                                  [logivia-dev system:authenticated]
$ kubectl --kubeconfig=lea.kubeconfig get pods
Error from server (Forbidden): pods is forbidden: User "lea" cannot list resource "pods" in API group "" ...
```

Authentifiée ne veut pas dire autorisée : sans RBAC, Léa n'a **aucun droit**.

RBAC : sujets, liaisons, rôles



Le rôle des développeurs TrackR

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: trackr-dev, namespace: trackr}
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log", "services", "configmaps", "events"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments/scale"]
  verbs: ["get", "patch", "update"]
```

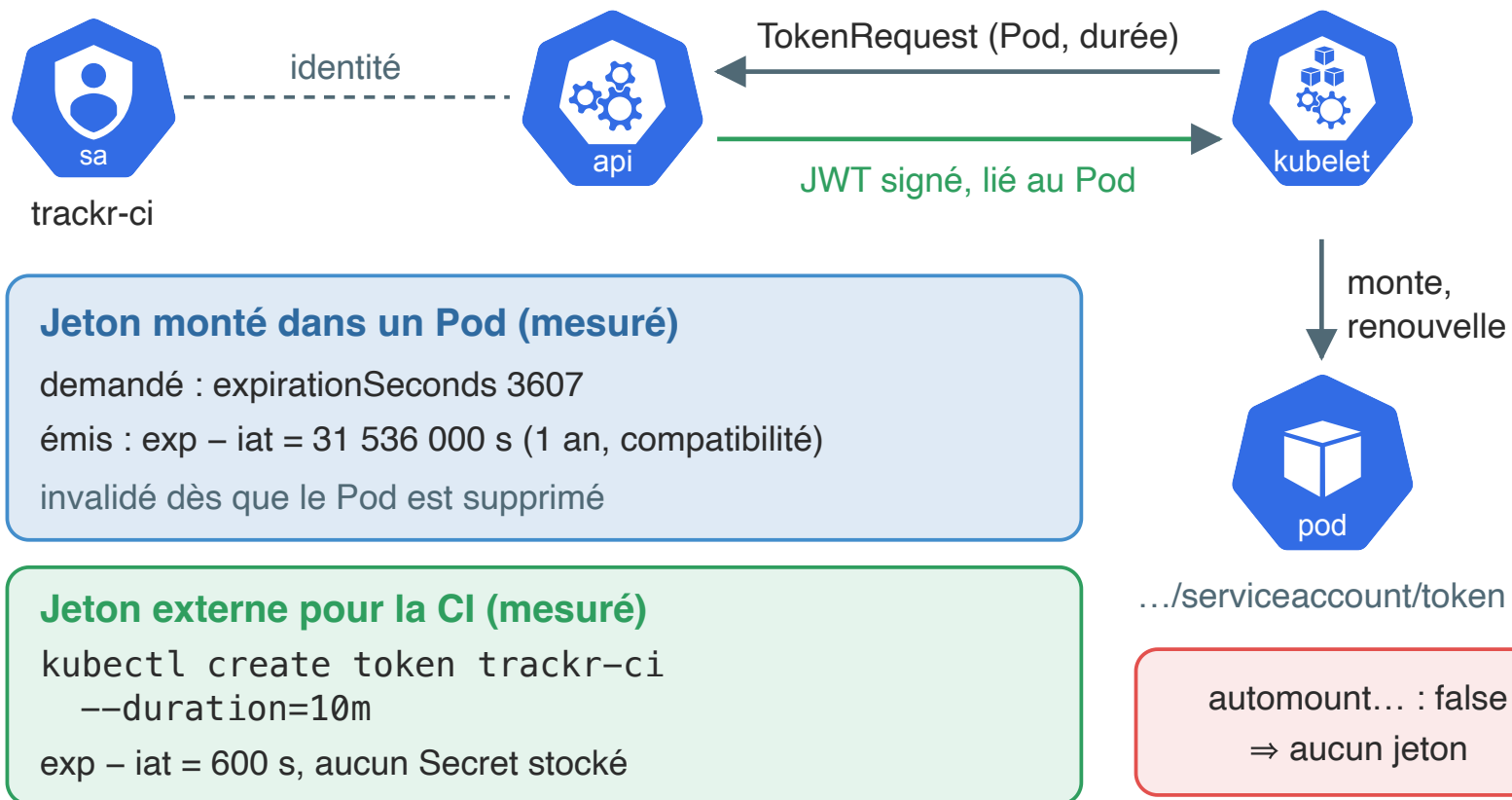
Le RoleBinding `trackr-dev` lie ce rôle au **groupe** `logivia-dev` (champ `0` du certificat). Verbes usuels : `get`, `list`, `watch`, `create`, `update`, `patch`, `delete`.

Tester : auth can-i et --as

```
$ kubectl --kubeconfig=lea.kubeconfig get pods
NAME                                READY   STATUS    RESTARTS   AGE
trackr-api-75c69bd9bb-8xgfc        1/1     Running   0           102s
...
$ kubectl --kubeconfig=lea.kubeconfig get secrets
Error from server (Forbidden): secrets is forbidden: User "lea" cannot list resource "secrets" ...
$ kubectl --kubeconfig=lea.kubeconfig exec deploy/trackr-api -- id
error: unable to upgrade connection: ... User "lea" cannot create resource "pods/exec" ...
$ kubectl auth can-i patch deployments --subresource=scale --as=lea --as-group=logivia-dev -n trackr
yes
$ kubectl auth can-i list pods --as=lea -n trackr                # sans le groupe : aucun droit
no
```

L'administrateur vérifie les droits **sans** le certificat de Léa grâce à `--as` et `--as-group` (impersonation). Pour une ressource globale (nœuds), il faut une ClusterRole et un ClusterRoleBinding.

ServiceAccounts et jetons projetés



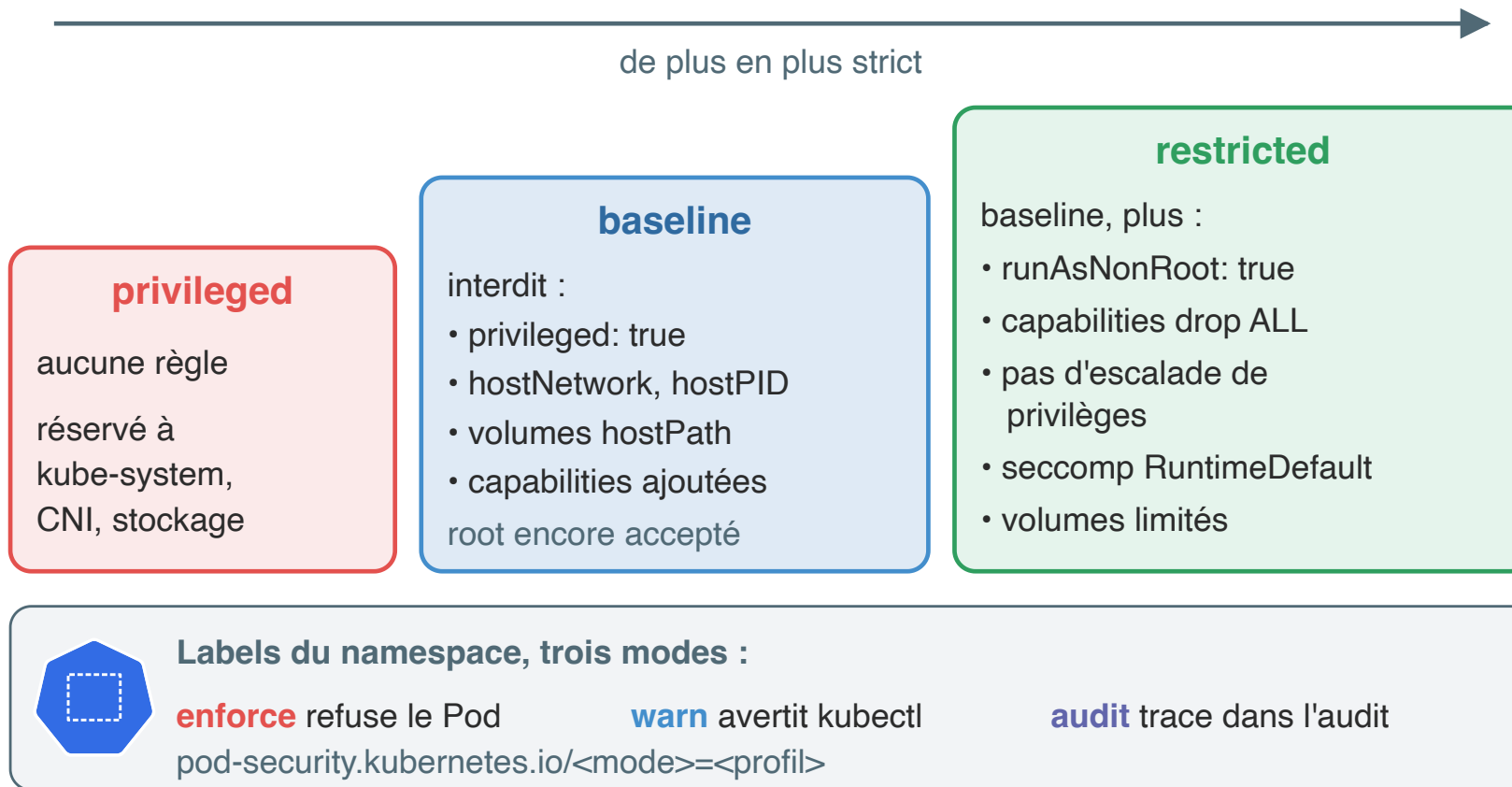
Moindre privilège : les réflexes

À éviter	À faire
cluster-admin pour tout le monde	un Role par équipe et par namespace
lier des personnes une à une	lier des groupes (O= du certificat, OIDC)
verbes et ressources « * »	verbes et ressources listés
jeton monté dans chaque Pod	automountServiceAccountToken: false
jeton éternel stocké dans un Secret	kubectl create token --duration=10m

Le jeton de la CI `trackr-ci` peut faire `rollout restart`, mais reçoit `Forbidden` sur `delete deploy` et `get secrets` (mesuré).

3. Admission et Pod Security

Les trois profils Pod Security Standards



Pod Security Admission en action

```
kubectl label --overwrite ns trackr \  
pod-security.kubernetes.io/enforce=restricted \  
pod-security.kubernetes.io/enforce-version=v1.35 \  
pod-security.kubernetes.io/warn=restricted \  
pod-security.kubernetes.io/audit=restricted
```

```
$ kubectl -n trackr run root-pod --image=nginx:1.27-alpine  
Error from server (Forbidden): pods "root-pod" is forbidden: violates PodSecurity "restricted:v1.35":  
  allowPrivilegeEscalation != false (...), unrestricted capabilities (...),  
  runAsNonRoot != true (...), seccompProfile (...)
```

- Un **Deployment** non conforme est créé, mais son ReplicaSet échoue (`FailedCreate`).
- Avant d'activer : `kubectl label --dry-run=server ...` liste les Pods existants en infraction.

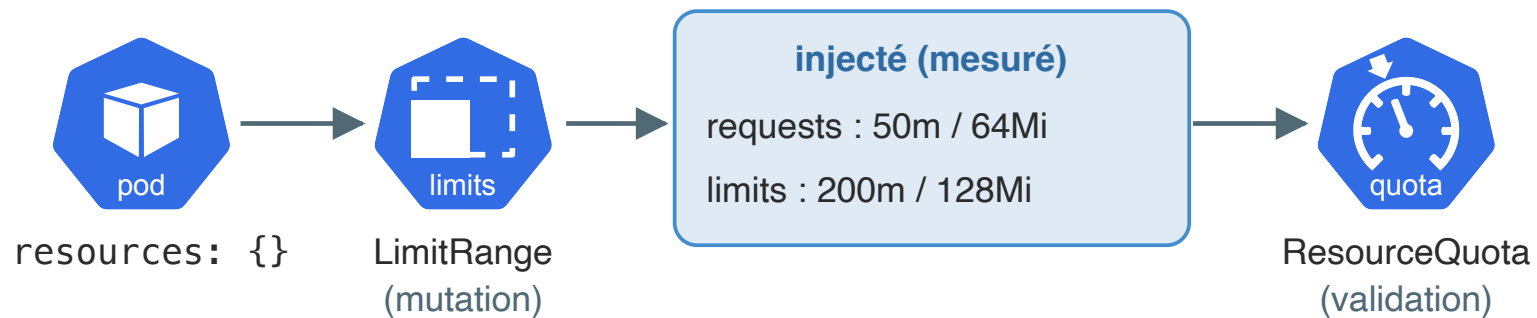
Un securityContext conforme « restricted »

```
spec:
  automountServiceAccountToken: false
  securityContext:                                # niveau Pod
    runAsNonRoot: true
    runAsUser: 100                                # utilisateur "app" de l'image podinfo
    seccompProfile: {type: RuntimeDefault}
  containers:
    - name: podinfo
      securityContext:                              # niveau conteneur
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: {drop: ["ALL"]}
        volumeMounts: [{name: tmp, mountPath: /tmp}] # emptyDir inscriptible
```

Mesuré : `touch /pirate` → Read-only file system. Piège : une image root (nginx) avec `runAsNonRoot` passe PSA, puis échoue en `CreateContainerConfigError`.

4. Ressources, réseau et etcd

ResourceQuota et LimitRange



Trois refus mesurés sur le lab

Quota sans LimitRange

Pod sans resources :

« must specify
limits.cpu... »

Borne max dépassée

limits.cpu: 1 :

« maximum cpu usage
per Container is 500m »

Plafond atteint

9e Pod du namespace :

« exceeded quota:
pods=8 »

NodePort : aussi
refusé (quota 0)

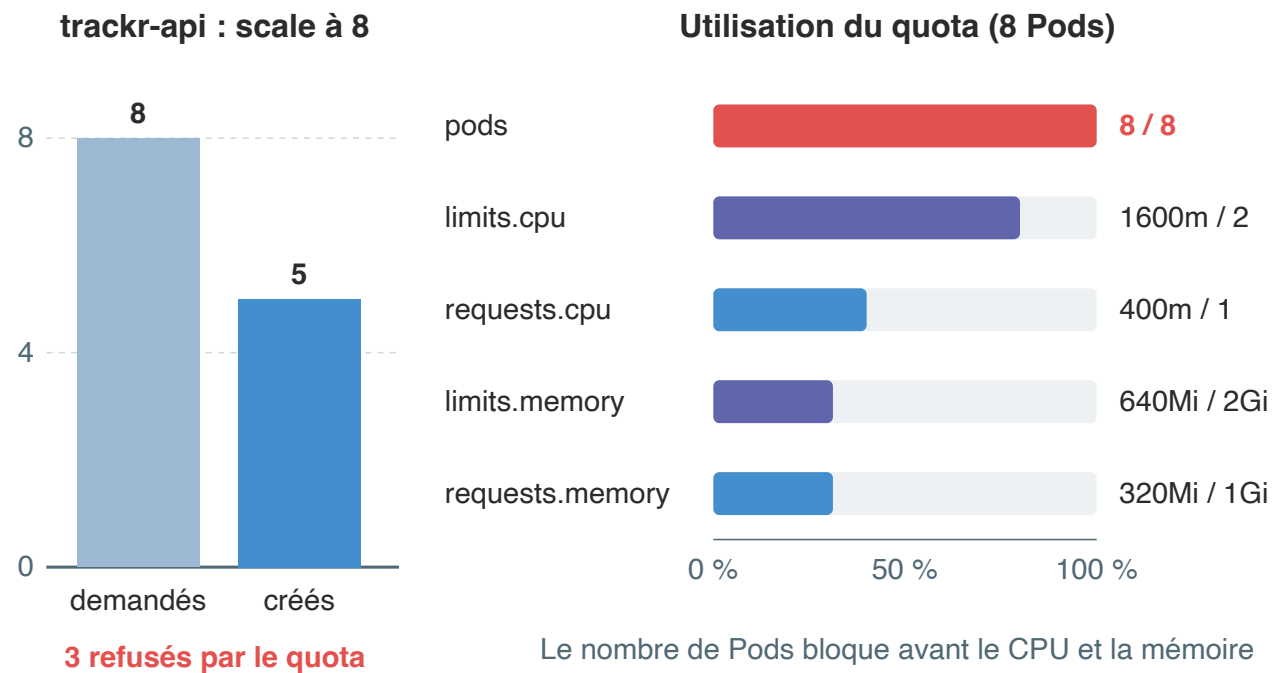
Plafonds et valeurs par défaut de TrackR

```
apiVersion: v1
kind: ResourceQuota
metadata: {name: trackr-quota}
spec:
  hard:
    pods: "8"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
    services.nodeports: "0"
```

```
apiVersion: v1
kind: LimitRange
metadata: {name: trackr-limits}
spec:
  limits:
    - type: Container
      defaultRequest: {cpu: 50m, memory: 64Mi}
      default: {cpu: 200m, memory: 128Mi}
      min: {cpu: 10m, memory: 16Mi}
      max: {cpu: 500m, memory: 512Mi}
```

Le quota **plafonne la somme** du namespace ; le LimitRange **complète et borne** chaque conteneur. Les deux vont ensemble : sans LimitRange, un Pod sans `resources` est refusé.

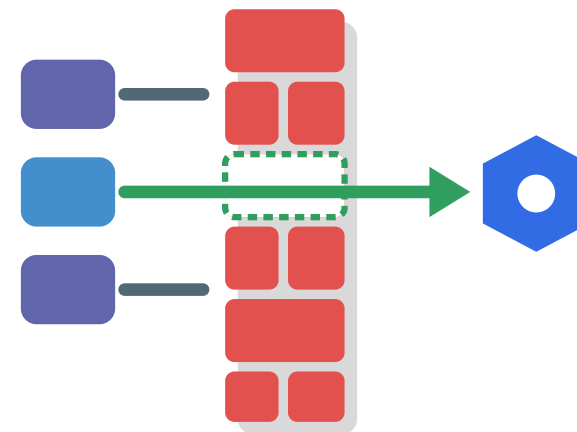
Quota dépassé : la mesure



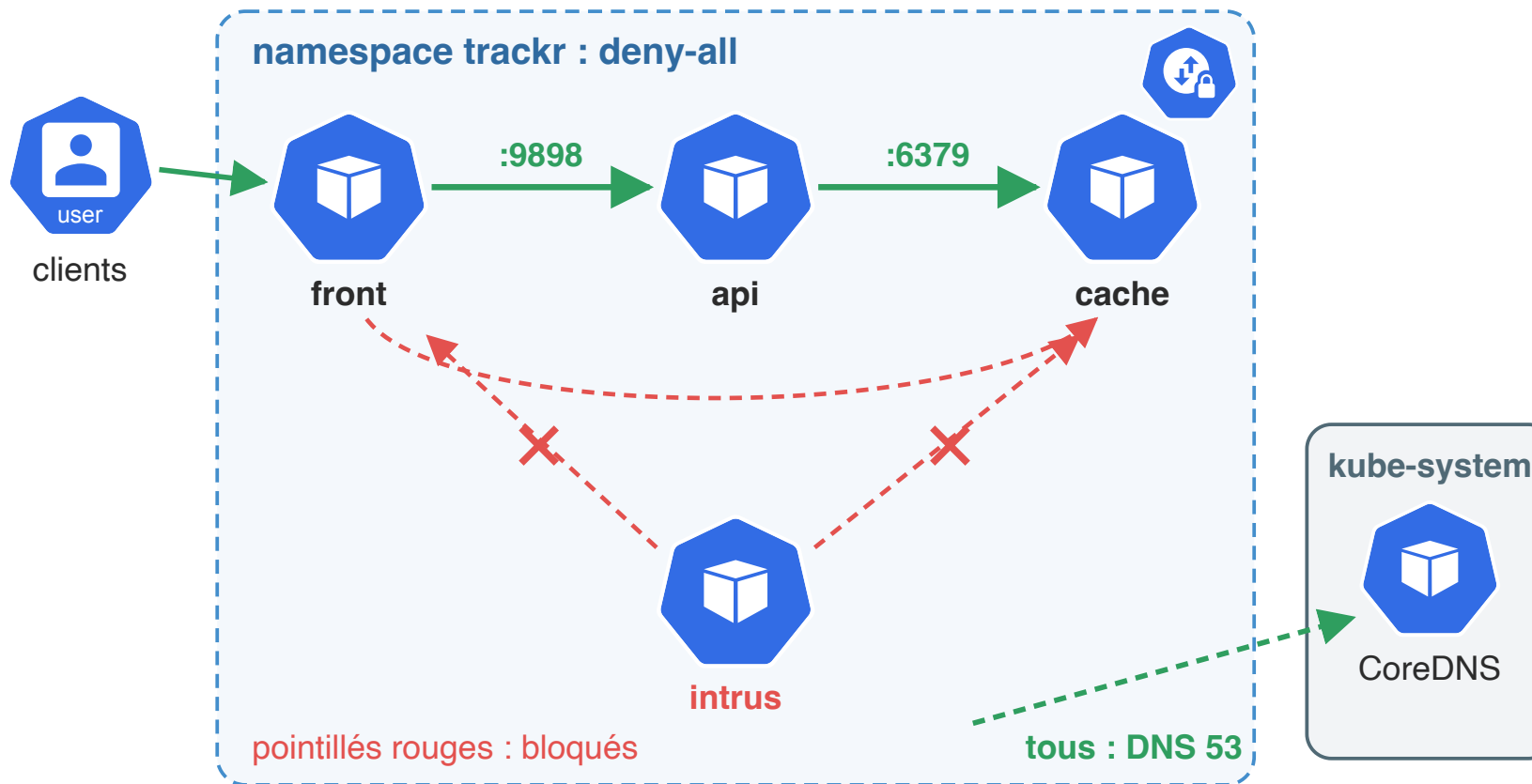
Mesuré sur le lab (kind v1.35, 3 nœuds) : `scale --replicas=8` → `READY 5/8`.

NetworkPolicy : les principes

- Une NetworkPolicy **sélectionne des Pods** et liste ce qui est **autorisé**.
- Pod sélectionné : tout le reste est **refusé** dans ce sens (Ingress, Egress).
- Les règles **s'additionnent** : pas de règle « deny » explicite.
- Un flux s'ouvre **des deux côtés** : egress de l'émetteur, ingress du récepteur.
- Il faut un CNI qui les applique : **kindnet** du lab le fait (testé).



TrackR cloisonné : front → api → cache



Tout fermer, puis ouvrir au plus juste

```
# 40-netpol-deny-all.yaml : tout fermer, en entrée et en sortie
spec: {podSelector: {}, policyTypes: ["Ingress", "Egress"]}
---
# 42-netpol-trackr.yaml (extrait) : la politique de l'API
spec:
  podSelector: {matchLabels: {app: trackr, tier: api}}
  policyTypes: ["Ingress", "Egress"]
  ingress:
    - from: [{podSelector: {matchLabels: {app: trackr, tier: front}}}]
      ports: [{protocol: TCP, port: 9898}]
  egress:
    - to: [{podSelector: {matchLabels: {app: trackr, tier: cache}}}]
      ports: [{protocol: TCP, port: 6379}]
```

Oubli classique : après `deny-all`, le **DNS** est coupé (`wget: bad address`). La politique `41-netpol-dns.yaml` rouvre l'egress UDP/TCP 53 vers CoreDNS pour tous les Pods.

Matrice de connectivité mesurée

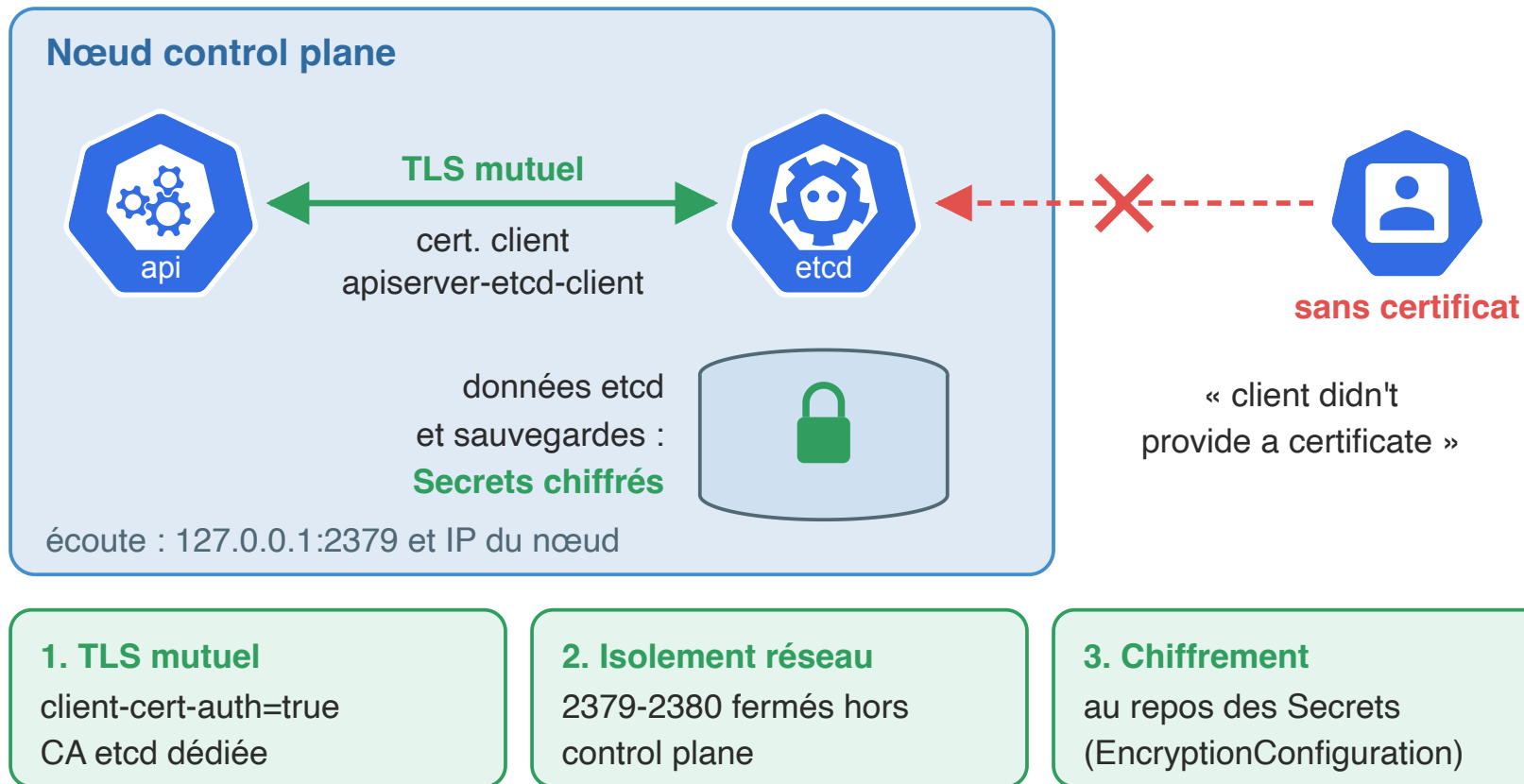
1. Sans NetworkPolicy					2. deny-all				
	front	api	cache	DNS		front	api	cache	DNS
front	0,2 s	0,2 s	0,2 s	0,2 s	front	5,2 s	5,2 s	5,3 s	3,3 s
api	0,2 s	0,2 s	0,3 s	0,2 s	api	5,2 s	5,3 s	5,2 s	3,2 s
cache	0,3 s	0,2 s	0,2 s	0,2 s	cache	5,2 s	5,4 s	5,3 s	3,2 s
intrus	0,2 s	0,2 s	0,2 s	0,2 s	intrus	20,4 s	20,3 s	20,5 s	3,2 s

3. deny-all + DNS					4. Ouvertures TrackR				
	front	api	cache	DNS		front	api	cache	DNS
front	3,2 s	3,2 s	3,2 s	0,2 s	front	3,2 s	0,2 s	3,2 s	0,2 s
api	3,7 s	3,3 s	3,3 s	0,2 s	api	3,2 s	3,2 s	0,2 s	0,2 s
cache	3,2 s	3,2 s	3,2 s	0,2 s	cache	3,2 s	3,2 s	3,3 s	0,2 s
intrus	3,3 s	3,2 s	3,2 s	0,2 s	intrus	3,2 s	3,2 s	3,5 s	0,2 s

ligne = Pod source · colonne = cible · vert : réponse · rouge : échec

Mesuré sur le lab (kindnet) : `wget -T 3`, `nc -w 3`, `nslookup` depuis chaque Pod ; un échec = délai expiré, paquet ignoré.

Restreindre l'accès à etcd



Chiffrer les Secrets au repos

```
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
- resources: ["secrets"]
  providers:
  - secretbox:
      keys:
      - name: key1
        secret: <32 octets aléatoires, base64>
  - identity: {} # relit les anciens Secrets
```

Option du manifeste statique de kube-apiserver :

`--encryption-provider-config`. Rejoué sur un cluster **jetable**, jamais sur le cluster partagé.



etcd avant et après chiffrement

Avant : Secret stocké en clair (extrait de hexdump -C)

```
/registry/secrets/trackr/trackr-redis
...k8s.....v1..Secret.....trackr-redis....
.....password..Logivia-S3cr3t-2026..0paque

etcdctl get /registry/secrets/... : le mot de passe se lit directement
```

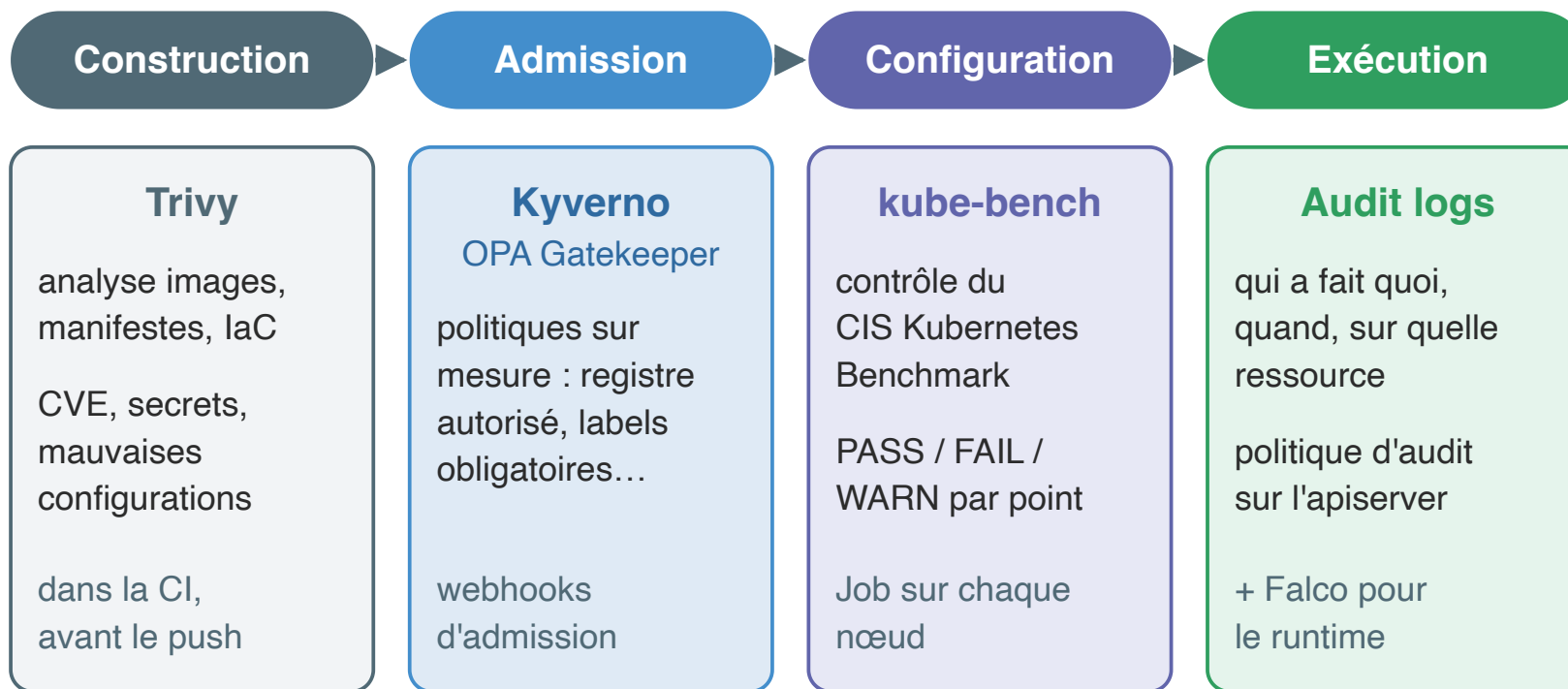
EncryptionConfiguration + redémarrage apiserver (42 s) ↓ puis kubectl replace des Secrets existants

Après : Secret chiffré par l'apiserver avant écriture

```
/registry/secrets/trackr/trackr-redis
k8s:enc:secretbox:v1:key1:~...{)..T.2.....UQ.....
grep -c Logivia → 0

kubectl get secret ... → Logivia-S3cr3t-2026 (déchiffré à la lecture)
```


Outils pour aller plus loin



Présentés pour information : non installés sur le lab de formation

Démo 7 et TP 7

Démo 7 — `demos/demo-07-rbac-netpol-etcd.md` : utilisatrice Léa par CSR, refus RBAC, matrice NetworkPolicy avant/après, Secret lu dans etcd puis chiffré (cluster jetable).

TP 7 — `tp/tp-07-securiser-trackr.md` : sécuriser le namespace TrackR avec RBAC, ServiceAccount, Pod Security `restricted`, quotas et NetworkPolicy. Durée : 50 min.

Exercice 7 (facultatif) — `exercices/exercice-07-rbac.md` : droits d'un ServiceAccount de supervision.

À retenir

- Chaque requête traverse **authentification** → **RBAC** → **admission** ; tout refus renvoie `Forbidden`.
- RBAC est **additif et fermé par défaut** : lier des groupes à des Roles minimaux.
- Jetons **courts** et **non montés** quand le Pod n'appelle pas l'API.
- Namespace en **restricted** : non root, `drop ALL`, seccomp, pas d'escalade.
- **Quota + LimitRange** ensemble ; **deny-all** puis ouvertures, sans oublier le DNS.
- etcd : **TLS mutuel**, réseau isolé, **Secrets chiffrés** au repos.